Assignment 3: Inheritance, Polymorphism & Abstraction (Practical)

Practical Coding Exam: 2 Architectural Challenges (Easy & Medium)

Total Questions: 2 Practical	**Difficulty: Easy & Medium**	**Format: Hands-on Source Code**	**Compiler: JDK 17+**

Submission & Execution Rules:
• All solutions must be compilable, working Java source files (.java) executing cleanly with javac and java.
• Validate user input, avoid unhandled runtime exceptions, and provide formatted console output as demonstrated.
• Each question must contain clear comments explaining the chosen algorithm and data structure.

[EASY LEVEL] — Shape Hierarchy with Dynamic Method Dispatch PRACTICAL LAB

Problem Statement: Implement an extensible shape calculation engine utilizing abstract classes, method overriding with @Override, and runtime dynamic method dispatch.

Technical Requirements & Implementation Checklist:

• Abstract Class: abstract class Shape with private String color, concrete constructor, getter, and abstract methods double calculateArea() and double calculatePerimeter().

• Subclasses: Circle(String color, double radius) and Rectangle(String color, double width, double height) extending Shape.

• Dynamic Dispatch Runner: In ShapeRunner.java, instantiate an array of polymorphic references: Shape[] shapes = new Shape[]{ new Circle("Red", 5.0), new Rectangle("Blue", 4.0, 6.0), ... };.

• Iterate through the array with an enhanced for-loop, dynamically invoking calculateArea() and calculatePerimeter(), and printing results in a clean table.

Starter Code Template:

```
public abstract class Shape {
    private String color;
    public Shape(String color) { this.color = color; }
    public String getColor() { return color; }
    public abstract double calculateArea();
    public abstract double calculatePerimeter();
}
```

Sample Console Interaction:

```
--- POLYMORPHIC SHAPE RENDERING ---
Shape 1 [Circle - Red]       Area:  78.54 | Perimeter: 31.42
Shape 2 [Rectangle - Blue]   Area:  24.00 | Perimeter: 20.00
Total Accumulated Area: 102.54
```

Evaluation Criteria: *Abstract class and constructor design | Subclass overriding & formula accuracy | Polymorphic array iteration & dynamic dispatch*

[MEDIUM LEVEL] — Employee Payroll System with Multiple Interfaces & Safe Casting PRACTICAL LAB

Problem Statement: Architect an employee payroll processing hierarchy demonstrating multiple interface implementation, compile-time vs runtime polymorphism, and safe downcasting with instanceof.

Technical Requirements & Implementation Checklist:

• abstract class Employee: fields (id, name, basePay), abstract calculateMonthlySalary(), implements Comparable to sort employees by total compensation descending.

• Interface Taxable: method double calculateTax() (15% for salaries > $5,000, 10% otherwise).

• Interface BenefitsEligible: method String getHealthcarePlan().

• Concrete Classes: FullTimeEmployee (implements Taxable & BenefitsEligible; gets 20% bonus), Contractor (implements Taxable only; paid by hourly rate * hours).

• Downcasting Guard: In PayrollManager.java, loop through employees, invoke polymorphic methods, and use instanceof pattern matching to safely downcast and print benefits ONLY for eligible employees without throwing ClassCastException.

Starter Code Template:

```
public abstract class Employee implements Comparable<Employee> {
    protected String id, name; protected double basePay;
    public Employee(String id, String name, double basePay) { /* TODO */ }
    public abstract double calculateMonthlySalary();
    // TODO: implement compareTo
}
```

Sample Console Interaction:

```
Processing Payroll (Sorted by Highest Compensation):
1. Alice (Full-Time)  | Pay: $7,200.00 | Tax: $1,080.00 | Benefits: Executive Health Plan
2. Bob (Contractor)   | Pay: $4,500.00 | Tax: $450.00   | Benefits: None (Contractor)
Safe downcast verified: No ClassCastException thrown!
```

Evaluation Criteria: *Abstract class & interface hierarchy | Safe downcasting with instanceof pattern | Comparable sorting & payroll report*